Trekking Management Application V2 — Project Report

1. Student Details

Field	Details
Name	Damodara Manish
Roll Number	24f2005363
Email	24f2005363@ds.study.iitm.ac.in

About Me:

I am a student of the IIT Madras BS Degree Program with a strong interest in full-stack web development and backend systems. I built this project to automate trekking activity management end-to-end, covering trek route creation, staff assignment, trekker bookings, status tracking, background jobs, and caching using modern web technologies.

2. Project Details

Project Title: Trekking Management Application V2 — Role-Based Trekking Activity Management System

Problem Statement

Trekking organizations traditionally manage trek routes, staff assignments, and trekker bookings manually through spreadsheets and phone/email coordination. This leads to booking errors, poor slot tracking, and inefficient communication between admins, staff, and trekkers. The goal was to build a full-stack web application that automates the entire trekking activity lifecycle, from route creation to booking and completion.

Approach

The application was developed using a RESTful API architecture (Flask + Vue 3) with three role-based workflows — Admin, Staff, and Trekker — supported by background job processing and caching:

- Authentication & Role Management: Unified User model with Flask-Security session-based auth, role-based access control (@roles_required), single auto-seeded admin, trekker self-registration, and login guard for blacklisted/inactive accounts.
- Admin Module: Dashboard with overview stats; trek route CRUD with staff assignment and status updates (Pending/Open/Closed/Cancelled/Ongoing/Completed); staff management

(create, search, block/unblock); client management (search, block/unblock, delete); booking overview; monthly report trigger via Celery.

- Staff Module: Dashboard with assigned treks and participant counts; trek detail with participant list; status updates (Open/Closed/Ongoing); mark trek complete; slot adjustment; profile and password management.
- Trekker Module: Home summary, Explore treks with filters (difficulty/location/duration), trek detail and booking, My Trips with cancellation and CSV export, History view, and account/password management.
- Background Jobs (Celery): Daily reminder emails (9:00 AM) for treks starting the next day, monthly HTML summary report (1st of month, 7:00 AM) to admin, on-demand CSV export of booking history with task polling, and async test email pipeline verification.
- Caching: Redis-backed caching with 30-second TTL on frequently accessed GET routes (admin stats, trek/staff/client lists, bookings, trekker explore and trek detail), auto-cleared on write operations via an after_request hook, with graceful degradation if Redis is unavailable.

3. AI / LLM Declaration

Perplexity AI was used throughout the project as a learning and debugging assistant. The extent of usage is approximately 10-15%, covering the following areas:

- Guidance on Flask-Security session-based authentication setup and role configuration.
- Celery task scheduling setup (daily reminder, monthly report) and Redis broker configuration for async jobs.
- Redis caching strategy — 30-second TTL design and cache invalidation via after_request hook.
- Debugging SQLAlchemy relationship errors between User, Trek, Booking, and StaffInfo models.
- Code review and identifying logical bugs in slot adjustment and booking status transitions.

All final implementation logic, database schema design, API design decisions, and integration were done manually. AI was used as a guidance tool, not as a code generator for complete features.

4. Frameworks and Libraries Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	ORM for database models and queries
Flask-Security	Session-based authentication and role-based access control
SQLite	Lightweight relational database

Flask-Caching	Redis-backed caching for API response optimization
Redis	Cache store (30s TTL) and Celery message broker
Celery	Asynchronous and scheduled background task queue
smtplib (Gmail SMTP)	Email notifications (reminders, reports)
Vue 3 (Options API)	Frontend reactive SPA framework
Vue Router 4	Client-side routing for SPA views
Bootstrap 5	Responsive frontend styling
Fetch API	Frontend-backend HTTP communication

5. Database Schema / ER Diagram

Tables

- User — id, fs_uniquifier, email, password, name, mobile, active, blacklisted, created — managed by Flask-Security
- Role — id, name, desc
- role_assignments — user_id (FK -> User), role_id (FK -> Role) — many-to-many join table
- staff_info — id, user_id (FK -> User), info, exp_years, skills, state, created
- trek — id, title, place, level, duration, max_slots, left_slots, status, start_date, end_date, about, staff_id (FK -> staff_info), created
- booking — id, user_id (FK -> User), trek_id (FK -> trek), book_time, status, pay_status, qty, cancel_time

Relationships

- User to Role: Many-to-Many (via role_assignments join table)
- User to staff_info: One-to-One (each staff account has one staff profile)
- staff_info to trek: One-to-Many (a staff member is assigned to many treks)
- User to booking: One-to-Many (a trekker makes many bookings)
- trek to booking: One-to-Many (a trek receives many bookings)

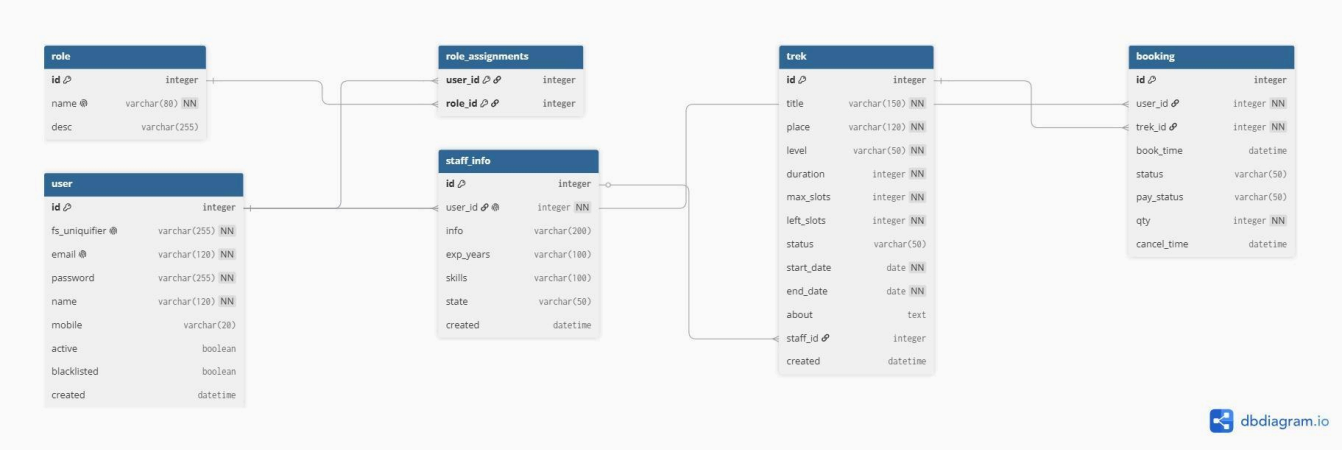


Figure 1: Entity-Relationship Diagram of the Trekking Management Application V2 database schema.

6. API Resource Endpoints

Authentication

Endpoint	Method	Access	Description
/api/sign-in	POST	Public	Login
/api/sign-out	GET	Any	Logout
/api/register	POST	Public	Trekker registration
/api/user-state	GET	Any	Current user session info

Admin APIs

Endpoint	Method	Access	Description
/api/adm-overview	GET	Admin	Dashboard stats (cached 30s)
/api/adm-treks/list	GET	Admin	List all treks (cached 30s)
/api/adm-treks/search	GET	Admin	Search treks
/api/adm-treks/add	POST	Admin	Create trek
/api/adm-treks/edit/<id>	PUT	Admin	Edit trek details
/api/adm-treks/update-state/<id>	POST	Admin	Change trek status
/api/adm-treks/remove/<id>	DELETE	Admin	Delete trek
/api/adm-guides/list	GET	Admin	List all staff (cached 30s)

/api/adm-guides/search	GET	Admin	Search staff
/api/adm-guides/add	POST	Admin	Create staff
/api/adm-guides/assign-route/<guide_id>/<route_id>	POST	Admin	Assign staff to trek
/api/adm-guides/block/<guide_id>	POST	Admin	Block staff
/api/adm-guides/unblock/<guide_id>	POST	Admin	Unblock staff
/api/adm-clients/list	GET	Admin	List all trekkers (cached 30s)
/api/adm-clients/search	GET	Admin	Search trekkers
/api/adm-clients/block/<client_id>	POST	Admin	Block trekker
/api/adm-clients/unblock/<client_id>	POST	Admin	Unblock trekker
/api/adm-clients/remove/<client_id>	DELETE	Admin	Delete trekker
/api/adm-bookings	GET	Admin	View all bookings (cached 30s)
/api/probe-mail	GET	Admin	Test email (sync)
/api/async-probe-mail	GET	Admin	Test email (async via Celery)
/api/trigger-monthly-report	GET	Admin	Trigger monthly report

Staff APIs

Endpoint	Method	Access	Description
/api/staff-panel/summary	GET	Staff	Dashboard stats
/api/staff-panel/routes	GET	Staff	List assigned treks
/api/staff-panel/routes/<id>	GET	Staff	Trek detail + participants
/api/staff-panel/routes/<id>/change-status	POST	Staff	Update trek status
/api/staff-panel/routes/<id>/adjust-slots	POST	Staff	Adjust available slots
/api/staff-panel/routes/<id>/mark-done	POST	Staff	Mark trek as completed
/api/staff-detail	GET	Staff	View own profile

/api/staff-detail	PUT	Staff	Update own profile
/api/staff-change-key	POST	Staff	Change password

Trekker APIs

Endpoint	Method	Access	Description
/api/trekker/home	GET	Trekker	Dashboard summary
/api/trekker/explore	GET	Trekker	Browse open treks (filterable, cached 30s)
/api/trekker/treks/<id>	GET	Trekker	Trek detail (cached 30s)
/api/trekker/reserve	POST	Trekker	Book trek seats
/api/trekker/cancel/<book_id>	POST	Trekker	Cancel booking
/api/trekker/my-trips	GET	Trekker	Active bookings
/api/trekker/history	GET	Trekker	Completed/cancelled history
/api/trekker/update-profile	PUT	Trekker	Update name/mobile
/api/trekker/change-passcode	POST	Trekker	Change password
/api/tasks/export-csv	GET	Trekker	Trigger async CSV export
/api/tasks/status/<task_id>	GET	Any	Check Celery task status

Health

Endpoint	Method	Access	Description
/api/health-pulse	GET	Any	Server health check
/api/db-audit	GET	Any	Database connection check

7. Background Jobs (Celery)

Task	Schedule	Description
Daily Reminder	Every day 9:00 AM	Sends email to trekkers about treks starting tomorrow
Monthly Report	1st of month, 7:00 AM	Generates HTML summary (treks, participants, popular treks) and emails admin

CSV Export	On-demand	Exports user's booking history as CSV, polls for status, shows download
Async Test	On-demand	Sends test email via Celery to verify async pipeline

8. Security

- Session-based authentication via Flask-Security
- Role-based access control enforced on all routes
- Blacklisted/inactive users blocked at login
- Only assigned staff can manage a given trek
- Password hashing with bcrypt
- CSRF disabled (session-based API)
- No hardcoded credentials in source code (all in config.py)

9. Video Presentation

Drive Link:
https://drive.google.com/file/d/17tefmfj9pC9q0b93En7z765LPKLmzMlu/view?usp=sharing

— End of Report —